

Planung, Zeitmanagement und Durchführung einer Forschungsarbeit, speziell einer Bachelorarbeit

Studiengang Angewandte Bioinformatik
an der Technischen Hochschule Bingen

Dieses Dokument enthält *verbindliche* Instruktionen und Vorgaben, die zur Durchführung einer Bachelorarbeit eingehalten und umgesetzt werden sollen. Das Dokument dient den Studierenden zur Orientierung und als Hilfestellung, um typische Fehlerquellen zu vermeiden und erfolgreich die Bachelor-Phase umzusetzen.

Allgemeines

- Drei Meetings "**Kick Off**", "**Midterm**", und "**Endspurt**" sind *rechtzeitig* zu vereinbaren. Um Probleme bei der Terminfindung zu vermeiden, müssen diese Termine mit einigen Wochen Vorlauf geplant werden. Idealerweise wird eine Plattform zur Terminfindung genutzt und für jeden Termin drei bis maximal fünf Vorschläge gemacht. Jedes Treffen sollte auf 45 bis maximal 60 Minuten angesetzt sein. Bestenfalls sind diese Termine spätestens in der Mitte der Praxisphase erfolgreich und verbindlich vereinbart.
- Zumindest für die ersten beiden Meetings muss der Studierende bitte eine *kurze Präsentation* (PowerPoint oder ähnlich) zu den Inhalten vorbereitet haben. Die Präsentation für das Midterm-Meeting ist besonders unerlässlich!
- *Wöchentlich* sind Reports abzuliefern über den Stand der Dinge; für das Format dieser Reports siehe unten.
- Idealerweise ist bei den beiden ersten Meetings (Start, Midterm) der (direkte) betriebliche Betreuer mit anwesend.

Am Ende oder Anfang jedes Meetings sind 5 bis 10 Minuten "unter vier Augen" zu reservieren, um evtl. Probleme in der Firma bzw. Institution ansprechen zu können:

- Stimmt die Supervision?
- Erhalte ich ausreichend Betreuung?

Themen des Kick Off Meetings

- Was ist die Aufgabe / Fragestellung?
- Was ist das experimentelle Design? Wie will ich die wissenschaftliche Fragestellung beantworten?
- Was sind meine Daten (Material)?
- Das Kick Off Meeting muss vor Beginn oder in den ersten zwei Wochen der Bachelor-Phase stattfinden

- Ein Sitzungsprotokoll ist durch den Studierenden anzufertigen. Wichtigster Inhalt ist eine To-Do-Liste mit Prioritäten und Zeitplan.

Themen des Midterm Meetings

- Welche Ergebnisse sind erzielt worden?
- Was bedeuten diese Ergebnisse im Hinblick auf die Fragestellung?
- Wie kann man die Ergebnisse gut darstellen (Plots, Tabellen, Text)?
- Was kann man in der verbleibenden Zeit noch evtl. "nachbessern" bzw. zusätzlich analysieren?
- Wie sieht die inhaltliche Struktur der Arbeit aus?
- Termin sollte ca. in der 6. Woche sein
- Als Sitzungsprotokoll: Struktur der Arbeit und noch offene To Dos

Inhalte des Endspurt, also der letzten drei bis vier Wochen der Bachelorarbeit

- Eine Vorabversion der Arbeit ist in der 9. Woche abzugeben
- Der Studierende erhält dann zeitnah (spätestens zwei Wochen nach Abgabe) asynchron eine kommentierte Version der Vorabversion und der Studierende bearbeitet die Arbeit entsprechend in den verbleibenden Wochen.
- Es besteht die Möglichkeit für Rückfragen

Format für wöchentliche Protokolle

Kurzer Text mit folgenden Überschriften und den jeweiligen Inhalten. Es ist in wenigen Sätzen und so kurz wie möglich zu schreiben.

- **Was?** Was haben Sie in der aktuellen Woche getan? Welche "Todos" wurden bearbeitet?
- **Wo?** Wo sind die Unterlagen, der Programmcode, die Ergebnisdateien, wissenschaftlichen Plots, und vor allem die Laborbuch-Einträge zu finden?
- **Was fehlt noch? Nächste Schritte.** Welche Schritte gedenken Sie in der kommenden Woche durchzuführen?
- **Probleme?** Welche Probleme sind bei der Arbeit aufgetreten? Wo brauchen Sie unter Umständen Hilfe oder Rückmeldung?
- **Zeit?** Wie viel Arbeitszeit haben Sie auf die jeweiligen To-Dos verwendet?

Führen eines bioinformatischen Laborbuchs

Ein Laborbuch ist *verpflichtend* während der Bachelorarbeit zu führen und muss den Supervisoren ständig einsichtig sein. Jeden Tag notiert der Studierende hier die Arbeiten, die er durchgeführt hat, und die wissenschaftlich, also für die Bachelorarbeit relevant sind. Ein Studientag, also ein Weiterbildungs-Tag, an dem Literatur gelesen und studiert wird, z.Bsp. schlägt sich *nicht* im Laborbuch nieder.

Organisation von elektronischen Dateien

Das Laborbuch dient einer lückenlosen und genauen Beschreibung *aller* Arbeitsschritte, die zur Erzeugung der wissenschaftlichen Ergebnisse durchgeführt wurden. Es wird empfohlen für ein Forschungsprojekt, wie die Bachelorarbeit folgende Ordnerstruktur anzulegen:

- **Docs:** Ordner enthält das *Laborbuch*, die *Sitzungsprotokolle*, die *wöchentlichen Reports* und alle weiteren Dokumentationen. Später wird hier auch die Bachelorarbeit (BA), also das Dokument der BA, in diesem Ordner oder einem Unterordner "Thesis" hinterlegt.
- **Material:** Ordner enthält alle Daten, die von Dritten stammen, also nicht im Rahmen des Projekts erzeugt wurden
- **Methoden:** Ordner enthält die Programm-Scripte und allen Code, der erzeugt wurde, um die Daten zu analysieren, zu visualisieren (Plots) und zu interpretieren (statistische Tests etc.).
- **Ergebnisse:** Alle durch die Analyse des Materials oder von Zwischenergebnissen entstandenen Ergebnis-Daten.

Ordner können selbstverständlich Unterordner enthalten. Die obigen Ordner können auch sehr gerne mit ihren englischen Namen abgelegt werden.

Geheimhaltung

Sollten bestimmte Dateien bzw. deren Inhalte einer Geheimhaltungspflicht unterliegen, sind diese Dateien selbstverständlich entsprechend vertraulich zu behandeln. Allerdings sollte idealerweise mit der Firma bzw. Institution ein Backup über beispielsweise einen internen Git-Server abgesprochen werden.

Versionierung

Alle Inhalte der obigen digitalen Unterlagen zum Forschungsprojekt *müssen* idealerweise mit Git versioniert werden. Gerne kann dann GitHub oder GitLab als Backup verwendet werden. Für große Dateien, die mit Git nicht effizient verwaltet werden können, soll "data version control" (DVC) benutzt werden:

<https://dvc.org/>

Die Vergabe von "semantischen Versionsnummern" ist stark empfohlen:

<https://semver.org/lang/de/>

Format von Laborbuch-Einträgen

Um die Reproduzierbarkeit der wissenschaftlichen Ergebnisse zu gewährleisten, muss jeder einzelne Schritt, der Eingabedaten entgegen nimmt und Ausgabedaten erzeugt, lückenlos dokumentiert sein. Hierfür sollte der Studierende in einem Textdokument seiner Wahl (Word oder freie Alternative, Wiki, wie es GitLab oder GitHub verfügbar ist, Markdown, Latex, etc.) alle durchgeführten Arbeitsschritte vollständig dokumentiert entsprechend eintragen. Beispielsweise muss selbst das einfache Sortieren einer Tabelle nach einer ausgewählten Spalte klar hinterlegt sein.

Ein gutes Format für Einträge im digitalen Laborbuch ist Folgendes:

- Datum und Thema: An welchem Tag wurde welcher Arbeitsschritt durchgeführt mit welchem Ziel?
- Git-Version (Tag): Welche Version der digitalen Projektunterlagen passt zum Laborbuch-Eintrag?
- Eine kurze und vollständige Beschreibung dessen, was getan wurde. Sind Details hierfür z.Bsp. in für den aktuellen Schritt extra geschriebenen Scripts hinterlegt, reicht ein Verweis auf das Script bzw. der Kommandozeilen-Aufruf dieses Scripts (siehe Beispiel unten).

Laborbuch Beispiel

Datum: 30.10.2023

git-tag: v0.1.2

Herunterladen der SwissProt Datenbank von UniProtKB in der *Version XYZ* mittels folgendem Kommando

```
wget
https://ftp.uniprot.org/pub/databases/uniprot/current_release/knowledgebase/complete/uniprot_sprot.fasta.gz
```

Danach Entpacken mit folgendem Kommando

```
gunzip uniprot_sprot.fasta.gz
```

Herausfiltern aller Protein-Identifizierer (Namen) von "Globin-Proteinen" mit folgendem Kommando

```
grep -P "^>" uniprot_sprot.fasta | grep -i "globin" >
uniprot_sprot_globins_IDs.txt
```

Datum: 31.10.2023

git-tag: v0.1.3

Herausfiltern der Globin-Aminosäure-Sequenzen mittels des dafür geschriebenen Python-Scripts `./methods/extract_sequences_by_identifiers.py` und dem Kommando

```
python3 ./methods/extract_sequences_by_identifiers.py
./results/uniprot_sprot_globins_IDs.txt
./material/uniprot_sprot.fasta
```

Guidelines bei der Softwareentwicklung

Code documentation

Jedweder Programmcode muss vollständig dokumentiert sein. Dokumentation muss in Standardformaten stattfinden, also z.Bsp. in PyDoc für Python, Roxygen für R, JSDoc für Javascript etc.

Test Driven Development

Um die Reproduzierbarkeit wissenschaftlicher Ergebnisse und die Korrektheit von Programmcode zu gewährleisten, sind die Prinzipien von Testgetriebener Softwareentwicklung anzuwenden. Hierbei sollen zumindest kritische Funktionen mit automatischen Tests auf Korrektheit überprüft werden. Siehe:

- https://de.wikipedia.org/wiki/Testgetriebene_Entwicklung
- https://en.wikipedia.org/wiki/Unit_testing
- <https://agilemanifesto.org/>

Lemmas

[Keep it simple and safe](#)

Always aim at implementing the simplest possible solution. Simple is robust, simple is easy to understand and thus helps working in collaboration, simple is thus less work to maintain. Consider the art of not doing work that is not needed as another interpretation of this lemma.

[Don't repeat yourself](#)

If you write two code blocks that look alike, move them into a separate function. If repeated code has a bug, the bug has to be fixed at all positions. Code repetition causes longer source files. And just statistically speaking the more code the likelier to have bugs. Repeated code is harder to understand due to the length, thus not good for collaborative work.

[Do one thing and do it well](#)

If you have a long piece of code, maybe a function, that does many things (steps), each of these steps can be moved into a separate function. This enables writing unit tests for each step. It enables reusing these functions should new code need to. It helps keep the code readable, robust, easier to maintain, and better to work on collaboratively.

[Use code generators](#)

Eric Raymond wrote the popular "The art of programming" book. He proposes a set of *very* help- and insightful programming lemmas to follow in order to produce code of high quality, i.e. easy to understand, robust, easy to maintain, and versatile. Click the link under the above section headline to read them all - and please *do* read them all. This lemma is one of the most important ones:

"Write [abstract programs that generate code](#) instead of writing code by hand."